

# **Práctica 1**

## **Microservicios: API REST en contenedores**

Sistemas Informáticos — Curso 2026/2027  
Escuela Politécnica Superior — Universidad Autónoma de Madrid

### **Autores**

Manuel Salvador Mayor  
Alejandro González García

**Grupo:** 1311

**Equipo:** 2

6 de octubre de 2026

## 1. Objetivo y arquitectura

Se ha desarrollado un servicio de almacenamiento compartido de documentos de texto dividido en dos microservicios Quart. El servicio de usuarios escucha en el puerto 5050 y administra identidades, contraseñas y tokens. El servicio de documentos escucha en el puerto 5051 y administra el directorio asociado a cada UID.

Los servicios se ejecutan en contenedores independientes coordinados mediante un único docker-compose.yml. Ambos reciben el mismo SECRET\_UUID como variable de entorno, mientras que el cliente desconoce ese secreto.

### Flujo de autenticación

Al crear un usuario se genera un UID aleatorio mediante uuid4. La contraseña se almacena como SHA-256 y el token se deriva con uuid5 a partir del UID y del secreto compartido. En las operaciones protegidas el cliente envía Authorization: Bearer <token> y el servicio de documentos recalcula el token esperado para el UID indicado en la ruta.

```
Authorization: Bearer <token>
```

### Persistencia

La persistencia es de libre elección según el enunciado. En este prototipo los usuarios y documentos se conservan en estructuras en memoria, por lo que se reinician al recrear los procesos.

## 2. Servicio de usuarios

user.py implementa la creación, el inicio de sesión y la modificación autenticada de la contraseña. Se rechazan usuarios duplicados, credenciales incorrectas, cabeceras inválidas y cuerpos JSON incompletos o con tipos incorrectos mediante códigos HTTP 400, 401 y 409.

| Método | Ruta  | Descripción                                  |
|--------|-------|----------------------------------------------|
| PUT    | /user | Crea un usuario y devuelve uid y token       |
| POST   | /user | Valida credenciales y devuelve uid y token   |
| PATCH  | /user | Cambia la contraseña del usuario autenticado |

## 3. Servicio de documentos

file.py mantiene un directorio lógico por UID. Los documentos se crean como privados. El propietario puede crear, actualizar, listar, borrar y cambiar su visibilidad. Un tercero no puede listar ni modificar el directorio; solamente puede recuperar un documento marcado como público conociendo el UID y el nombre.

| Método | Ruta                   | Descripción                          |
|--------|------------------------|--------------------------------------|
| GET    | /file/<uid>            | Lista los documentos del propietario |
| PUT    | /file/<uid>/<filename> | Crea o reemplaza el contenido        |
| GET    | /file/<uid>/<filename> | Recupera un documento permitido      |

| Método        | Ruta                   | Descripción                 |
|---------------|------------------------|-----------------------------|
| <b>DELETE</b> | /file/<uid>/<filename> | Elimina un documento propio |
| <b>PATCH</b>  | /file/<uid>/<filename> | Cambia el campo public      |

Las operaciones PUT, GET, PATCH y DELETE mantienen el comportamiento idempotente requerido. También se validan tokens, campos obligatorios y el tipo booleano de public.

## 4. Despliegue con Docker

Cada servicio dispone de su Dockerfile basado en python:3.12-slim. Las dependencias se instalan desde requirements.txt y Compose publica los puertos 5050 y 5051. La siguiente captura muestra la construcción correcta de ambas imágenes.

```

docker-compose build

#18 exporting layers 0.0s done
#18 exporting manifest sha256:837e942341850f7fb93074e272ef6c61a478ca14a9801e692680c59703038c52 done
#18 exporting config sha256:e815ca29b38e23e936c444394cafb621a0dd424d944855fc1740b1c4b7a104ec done
#18 exporting attestation manifest
sha256:5edd0218806cb6c5bd2c717b19995e23291a2b6da969ab2a5f89628c716a7e5c 0.1s done
#18 exporting manifest list sha256:40dc6e38369d3d99c612a14296dceaf7ce5a511cf29cab6f2c4fcc61c2fbf60c 0
done
#18 naming to docker.io/library/sistemas-informaticos-p1-user_service:latest 0.0s done
#18 unpacking to docker.io/library/sistemas-informaticos-p1-user_service:latest 0.0s done
#18 DONE 0.4s
#17 [file_service] exporting to image
#17 naming to docker.io/library/sistemas-informaticos-p1-file_service:latest 0.0s done
#17 unpacking to docker.io/library/sistemas-informaticos-p1-file_service:latest 0.0s done
#17 DONE 0.4s
#19 [user_service] resolving provenance for metadata file
#19 DONE 0.1s
#20 [file_service] resolving provenance for metadata file
#20 DONE 0.0s
Image sistemas-informaticos-p1-file_service Built
Image sistemas-informaticos-p1-user_service Built

```

Figura 1. Construcción de las dos imágenes Docker.

Tras ejecutar docker-compose up -d, los dos contenedores permanecen activos y cada puerto del host queda asociado al servicio correspondiente.

```

docker-compose ps

NAME                IMAGE                STATUS                PORTS
SERVICE            CREATED              STATUS                PORTS
si_file_service      sha256:019ffff01ca3d559490283c41eeb84b68ed782dd8bfc865e9e0920981f4becf2  "python
file.py"            file_service        11 minutes ago       Up 11 minutes       0.0.0.0:5051->5051/tcp, [::]:5051->5051/tcp
si_user_service      sha256:4d8daelb30ef426c5c196df1cf0b1d8b1193e0fce99ff7a69453e69fd0cd2dd0  "python
user.py"            user_service        11 minutes ago       Up 11 minutes       0.0.0.0:5050->5050/tcp, [::]:5050->5050/tcp

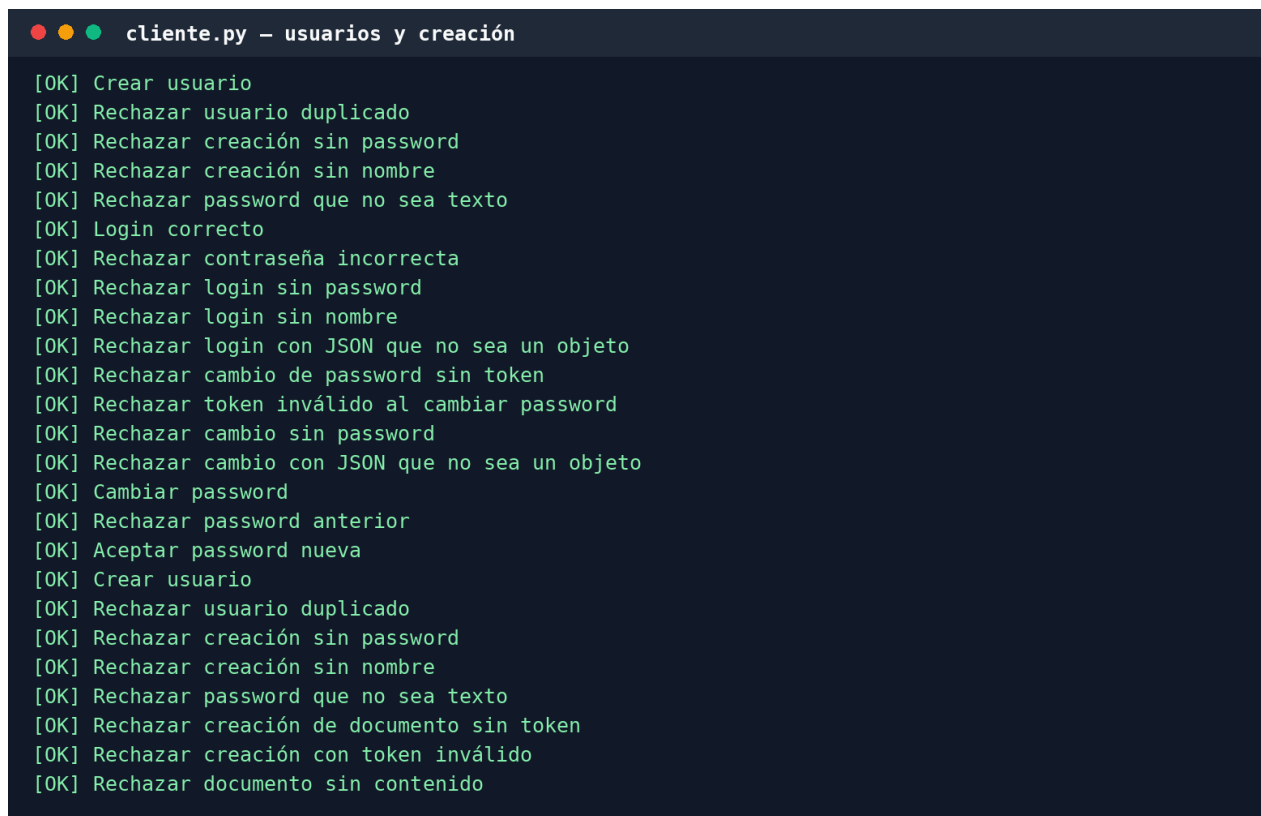
```

Figura 2. Estado de los contenedores y publicación de puertos.

## 5. Cliente de pruebas

cliente.py es un cliente no interactivo construido con requests. Utiliza nombres aleatorios para poder repetirse y ejecuta comprobaciones positivas y negativas sobre todas las rutas. Cada prueba muestra [OK] o [FALLO], valida el código HTTP y, cuando corresponde, el contenido JSON recibido.

Se cubren la creación duplicada, campos ausentes y tipos incorrectos, login erróneo, cambio de contraseña, tokens inválidos, permisos de terceros, documentos privados y públicos, actualización, listado y borrado.



```
cliente.py - usuarios y creación
[OK] Crear usuario
[OK] Rechazar usuario duplicado
[OK] Rechazar creación sin password
[OK] Rechazar creación sin nombre
[OK] Rechazar password que no sea texto
[OK] Login correcto
[OK] Rechazar contraseña incorrecta
[OK] Rechazar login sin password
[OK] Rechazar login sin nombre
[OK] Rechazar login con JSON que no sea un objeto
[OK] Rechazar cambio de password sin token
[OK] Rechazar token inválido al cambiar password
[OK] Rechazar cambio sin password
[OK] Rechazar cambio con JSON que no sea un objeto
[OK] Cambiar password
[OK] Rechazar password anterior
[OK] Aceptar password nueva
[OK] Crear usuario
[OK] Rechazar usuario duplicado
[OK] Rechazar creación sin password
[OK] Rechazar creación sin nombre
[OK] Rechazar password que no sea texto
[OK] Rechazar creación de documento sin token
[OK] Rechazar creación con token inválido
[OK] Rechazar documento sin contenido
```

Figura 3. Pruebas del servicio de usuarios y comienzo del servicio de documentos.

```
cliente.py - documentos y resultado

[OK] Rechazar contenido que no sea texto
[OK] Crear documento privado
[OK] Actualizar documento
[OK] Rechazar listado sin token
[OK] Rechazar listado de otro usuario
[OK] Listar documentos propios
[OK] Rechazar acceso anónimo a documento privado
[OK] Rechazar acceso de otro usuario a documento privado
[OK] Obtener documento privado
[OK] Rechazar cambio sin campo public
[OK] Rechazar visibilidad con JSON que no sea un objeto
[OK] Rechazar cambio de visibilidad sin token
[OK] Rechazar visibilidad que no sea booleana
[OK] Rechazar cero como visibilidad
[OK] Rechazar cambio de visibilidad por otro usuario
[OK] Hacer público el documento
[OK] Obtener documento público sin token
[OK] Volver a hacer privado el documento
[OK] Volver a impedir el acceso anónimo
[OK] Mantener acceso del propietario
[OK] Rechazar borrado por otro usuario
[OK] Eliminar documento como propietario
[OK] Documento eliminado no encontrado
[OK] Rechazar borrado de documento inexistente
[OK] Listado vacío después del borrado

Pruebas correctas: 50
Pruebas fallidas: 0
```

Figura 4. Pruebas de documentos, permisos, visibilidad y resultado final.

## 6. Resultado

La ejecución final completa 50 comprobaciones correctas y ninguna fallida. Las dos imágenes se construyen, los contenedores arrancan de forma independiente y el cliente puede comunicarse con ambos servicios mediante la API REST definida en el enunciado.